

Blueprint for a Pokémon Trading Card Ecosystem: An Integrated, Multi-Source Intelligence Platform for Vendors

Strategic Integration Framework for Multi-Platform Data Acquisition

The foundational challenge in designing an effective Pokémon card vendor intelligence system lies in creating a robust and resilient framework for acquiring data from a diverse and fragmented set of sources. The system's value proposition hinges entirely on its ability to aggregate, synthesize, and present timely information from multiple marketplaces, including eBay, TCGplayer, and Facebook Marketplace, while also identifying opportunities at physical events. This requires a multi-tiered integration strategy that prioritizes official Application Programming Interfaces (APIs) for reliability and efficiency, while strategically employing browser automation as a necessary fallback for platforms with limited or no programmatic access. This approach directly addresses the user's core need to eliminate manual data gathering tasks such as scrolling through listings and manually checking prices.

A critical case study within this framework is the eBay ecosystem, which exemplifies both the opportunities and complexities of modern API integration. The platform has undergone a significant transition, deprecating its legacy Finding and Shopping APIs in favor of a new suite of REST-based Buy and Browse APIs [41](#) [57](#). For any new integration, it is imperative to build upon this modern foundation. The eBay Buy API provides resources for searching items (`item_summary`), retrieving detailed item information (`getItem`), and managing orders (`GetOrders`) [20](#) [36](#) [37](#). Concurrently, the eBay Inventory API allows vendors to manage their stock by creating and updating inventory records before publishing them as live offers (`publishOffer`) [15](#) [61](#). These APIs form the bedrock of the system's automated market research and inventory tracking functionalities. Furthermore, eBay's Notification API is central to fulfilling the real-time alerting requirement. By enabling applications to create subscriptions for specific event types—such as `ItemListed`, `ItemRevised`, or `FixedPriceTransaction`—the system can receive push notifications instead of polling endpoints, dramatically reducing latency and server load [18](#) [92](#) [104](#)[105](#). These subscriptions can even be filtered to target

specific items or categories, adding a layer of precision to the alerting mechanism [113114](#). The authentication process, which involves OAuth2 and linking a developer account to a seller account, aligns perfectly with the user's request for a secure method to input API keys [6](#) [8](#) [9](#).

For TCGplayer, another major player in the Pokémon card space, a similar API-first strategy is paramount. While specific documentation was not provided in the source materials, TCGplayer's prominence suggests it offers a commercial API for accessing its extensive marketplace data, which includes valuable metrics like weighted median sale velocity [50](#). Integrating with this API would require a similar process of obtaining credentials and implementing a client to handle authenticated requests and data parsing. The success of the entire system depends on the quality and timeliness of the data pulled from these primary sources. Therefore, investing development effort into stable, well-documented APIs should be the highest priority.

When official APIs are unavailable, browser automation becomes an indispensable tool. The most prominent example of this necessity is Facebook Marketplace. Meta has announced the discontinuation of the Facebook Groups API, which previously provided a programmatic way to access content from community groups [117118](#). This forces developers to rely on headless browser automation to scrape data from Facebook's website. Tools like Playwright, which is designed for modern web automation, offer powerful features to handle anti-bot measures, intercept network traffic, and simulate realistic user interactions [93](#) [109](#). Building a dedicated scraper service using Playwright would be essential to fulfill the requirement of monitoring listings on Facebook Marketplace. However, this approach is inherently more fragile than using an API; website layout changes can break scrapers, and high-volume scraping can lead to IP bans or CAPTCHA challenges. It represents a trade-off between data accessibility and operational stability.

This leads to the third tier of the integration framework: identifying data gaps and setting realistic expectations. A key requirement was the ability to detect trending, uprising, and downtrending cards. The natural solution for this would be to use Terapeak, eBay's exclusive insights tool that analyzes supply, demand, and pricing data [30](#) [52](#). However, it is confirmed that Terapeak is not available as a public API; it is a feature within the eBay Seller Hub [51](#). Consequently, the system cannot simply "plug in" pre-calculated trend data. Instead, a custom analytics engine must be developed to perform this analysis. This engine would consume historical sales data obtained via the eBay Buy API and other available sources to build its own proprietary models for identifying market trends. This represents a significant but achievable engineering task.

Finally, the user expressed interest in connecting to official Pokémon inventory apps to pull a user's personal collection data. This goal appears highly speculative and likely unattainable. Official mobile applications are typically closed ecosystems with no public-facing APIs for third-party developers. Attempting to reverse-engineer such an app would be complex, legally questionable, and prone to breaking with updates. A more practical approach would be to implement a manual import/export functionality, allowing users to upload a simple text or CSV file containing their inventory list. This shifts the responsibility of data entry to the user but provides a viable workaround for integrating personal collections into the system's broader analysis. The table below summarizes the strategic integration approach for each target platform.

Platform / Tool	Primary Integration Method	Rationale & Key Considerations
eBay	Official REST APIs (Buy, Inventory, Notification)	Highest priority due to stability, reliability, and rich feature set (real-time notifications). Must navigate deprecation timelines and OAuth2 authentication. 15 40 41 92
TCGplayer	Official Commercial API	Essential for accessing deep marketplace data. Requires commercial agreement and API key management. Specifics depend on available documentation. 50
Facebook Marketplace	Browser Automation (Playwright/Selenium)	Necessary due to the discontinuation of the Facebook Groups API. More fragile and resource-intensive than API use; requires handling anti-bot measures. 93 117 118
Eventbrite	Official REST API	Direct fit for discovering upcoming trading shows. Allows for searching events, venues, and categories programmatically. 103
Official Pokémon Apps	Manual Import/Export (CSV/File)	Highly unlikely to have a public API. A manual workaround is the most feasible approach to incorporate user-owned inventory. Not explicitly covered in provided sources.

By adopting this tiered framework, the project can systematically address each data source according to its technical feasibility, balancing the pursuit of ideal API-based solutions with the pragmatic need for automation where APIs are absent or non-existent. This structured approach mitigates risk and ensures that development efforts are focused on achieving the most impactful results.

Architectural Blueprint for a Distributed Microservices Application

To effectively manage the complexity of aggregating data from numerous sources, processing it in various formats, and delivering real-time insights to a user interface, a monolithic architecture is fundamentally unsuitable. The system's requirements for high concurrency, fault tolerance, and independent scaling of specialized components

necessitate a distributed microservices architecture ³². This design decouples core functions—such as data ingestion, market analysis, and user interaction—into distinct, loosely coupled services. Each service can be developed, deployed, and scaled independently, leading to a more resilient, maintainable, and scalable system. This approach is particularly well-suited for the defined project, which involves a mix of I/O-bound tasks (API calls, browser automation) and CPU-intensive tasks (data analysis).

The cornerstone of this architecture is a message queue, such as RabbitMQ or Apache Kafka, which acts as the nervous system of the application ³¹. Producers, which include the specialized worker services for scraping and API consumption, publish raw data and events to specific queues. Consumers, such as the notification handler and the analytics engine, then subscribe to these queues and process the data asynchronously. This decoupling is critical for several reasons. First, it prevents slow or failing services from bringing down the entire system. If the analytics engine is temporarily overloaded, the data producers can continue to operate without being blocked. Second, it enables horizontal scaling; if one type of task, like scraping, becomes a bottleneck, additional instances of the scraping service can be spun up to process the queue faster. Third, it simplifies development and deployment, as teams can work on individual services without needing to coordinate tightly with others, provided they adhere to the contract defined by the message schema.

Within this model, several key microservices would be implemented:

1. **Frontend Service (React):** This service is responsible for rendering the user interface. It communicates with the backend primarily through a stateless API server, requesting dashboard data, sending user commands, and maintaining a persistent connection for receiving real-time updates. Its main concerns are UI logic, user experience, and data presentation.
2. **API Gateway:** Acting as the single entry point for all client requests, the API gateway handles routing, authentication, and rate limiting. It shields the internal microservices from direct external access, providing a unified and secure interface for the frontend and any future mobile clients.
3. **Authentication Service:** This service manages user accounts, sessions, and permissions. It securely stores sensitive information, including the API keys and OAuth tokens provided by users for integrated platforms like eBay and TCGplayer. This centralized management enhances security and simplifies credential handling across the system.

4. **Watchlist Management Service:** This service maintains user-defined watchlists of Pokémon cards. It provides endpoints for users to add, remove, and query their lists. When a price change or new listing matching a watchlist item is detected, the notification handler service would query this service to identify which users should be alerted.

5. **Specialized Worker Services:** These are the data ingestion engines of the system.

- **eBay Consumer Service:** Written in Python to leverage its data analysis libraries, this service interacts with the eBay Buy, Inventory, and Notification APIs. It fetches listing data, manages vendor inventories, and listens for push notifications from the eBay Notification API.
- **Scraping Service:** Built in Go for maximum performance and concurrency, this service uses Playwright to execute browser automation tasks against websites like Facebook Marketplace. Go's efficient goroutines are ideal for managing hundreds of simultaneous browser instances, making it well-suited for large-scale scraping operations [74 111](#).
- **Event Aggregator Service:** This service might be implemented in Node.js due to its suitability for high-concurrency I/O operations, allowing it to efficiently manage and orchestrate tasks for high-concurrency operations [75](#).

1. **Notification Handler Service:** This service consumes events from the message queue. Its role is to act as the system's central processing unit for incoming data. Upon receiving a new listing or price update, it would perform initial validation and then trigger downstream logic, such as comparing the new data against user watchlists or passing it to the analytics engine.

2. **Analytics Engine:** This is a batch-oriented service, likely written in Python, that runs periodically (e.g., daily or hourly). It consumes historical data from the database to perform trend analysis, identifying rising stars, over-saturated markets, and potential downturns. This service directly implements the custom "trend detection" feature required by the user.

3. **Database Layer:** A hybrid data storage strategy is recommended. A relational database like PostgreSQL is suitable for structured data, such as user profiles, watchlists, and application settings. For the vast amounts of semi-structured data generated by APIs and scrapers (e.g., JSON responses), a NoSQL database like MongoDB or Redis is more appropriate. Redis, in particular, is excellent for caching frequently accessed data and could be used to store recent prices for quick lookup. The following table outlines the proposed microservices architecture.

Service Name	Primary Language	Responsibility	Communication Mechanism
Frontend	React	User Interface Rendering	HTTP/API Gateway
API Gateway	Node.js	Request Routing, Authentication	N/A (Entry Point)
Auth Service	Node.js/Python	User Accounts, Credential Storage	Internal HTTP/RPC
Watchlist Service	Node.js/Python	Manages User Watchlists	Internal HTTP/RPC
eBay Consumer	Python	Consumes eBay APIs (Buy, Inventory, Notification)	Message Queue (Producer)
Scraping Service	Go	Browser Automation for Sites without APIs	Message Queue (Producer)
Notification Handler	Node.js/Python	Processes Events, Triggers Alerts	Message Queue (Consumer)
Analytics Engine	Python	Performs Trend Analysis on Historical Data	Database Polling/Batch Job
Database	PostgreSQL, MongoDB, Redis	Persistent Storage for Structured & Unstructured Data	Direct Access by Services

This distributed architecture provides a clear separation of concerns, allowing each component to be optimized for its specific task. The asynchronous nature enabled by the message queue is fundamental to achieving the real-time responsiveness required by the user, ensuring that the system remains performant even under heavy load from data ingestion tasks. It creates a robust and extensible foundation capable of accommodating new data sources and analytical features in the future.

Technology Stack Selection and Role Specialization

The selection of a technology stack is a critical decision that directly impacts the performance, scalability, and maintainability of the Pokémon card vendor intelligence system. The user's proposed combination of Go, Python, Bash, Node.js, and React is well-aligned with the project's multifaceted requirements, allowing for a polyglot approach where each language is chosen for its inherent strengths . This specialization avoids forcing a single language to perform tasks for which it is ill-suited, leading to a more efficient and robust final product.

Node.js emerges as the ideal choice for the core application server and several supporting services. Its non-blocking, event-driven I/O model makes it exceptionally efficient for handling a large number of concurrent connections, a common requirement for a real-time web application [75](#) . As the primary language for the API gateway and potentially the notification handler service, Node.js can manage the high volume of

requests from the React frontend and the asynchronous events flowing from the worker services with minimal overhead. Its vast ecosystem of libraries and frameworks, combined with the widespread expertise of JavaScript developers, further solidifies its position as a backbone technology for the backend [48](#).

Go (Golang) is selected for tasks that demand high performance, concurrency, and low-level system control. The most prominent use case for Go in this project is the **Scraping Service**. Web scraping, especially when performed at scale with headless browsers, is a computationally intensive task that benefits immensely from Go's lightweight goroutines, which enable thousands of concurrent operations far more efficiently than traditional OS threads [74](#). The open-source Lyrz Crawl project demonstrates Go's capability to achieve extremely high throughput, scraping thousands of pages per minute, making it a prime candidate for automating data collection from sites like Facebook Marketplace [111](#). A Go-based service can render web pages, execute JavaScript, and parse HTML with exceptional speed, minimizing the time-to-insight for newly listed items.

Python is chosen for its unparalleled strength in data science, machine learning, and scientific computing. The **Analytics Engine** and the **eBay Consumer Service** are the primary beneficiaries of Python's capabilities. The Analytics Engine will need to perform complex statistical analysis on historical sales data to identify market trends. Libraries like Pandas and NumPy provide the necessary tools for data manipulation and analysis, while scikit-learn could be used to build predictive models for identifying rising or falling card values. Similarly, the eBay Consumer Service will interact with APIs that return complex JSON payloads. Python's mature ecosystem of libraries simplifies the process of making authenticated API calls, parsing the responses, and transforming the data into a standardized format for the rest of the system. Python's readability and ease of use also make it an excellent choice for rapid prototyping and development of these analytical components [101](#).

Bash is included in the stack for its utility in orchestrating the overall workflow and managing infrastructure tasks. While not used for core application logic, Bash scripts can play a crucial role in the deployment and maintenance of the system. They can be used to automate the setup of development environments, manage container builds with Docker, orchestrate deployments to a Kubernetes cluster, and perform routine system administration tasks like log rotation and backups. Its role is more of a systems-level scripting language, ensuring that the entire application lifecycle—from development to production—is automated and repeatable.

React is the logical choice for the frontend, given its dominance in modern web development and its component-based architecture. A dashboard for a Pokémon card

vendor needs to be dynamic, responsive, and visually rich, displaying charts, tables, and real-time updates. React excels at building such user interfaces by efficiently updating only the parts of the DOM that have changed, leading to a smooth user experience 96. Its ecosystem includes powerful libraries for data visualization, such as Chart.js or D3.js, which are essential for presenting market trends and price histories in an intuitive way 69. The ability to build reusable UI components for elements like watchlist cards, alert modals, and search filters will significantly accelerate frontend development and ensure consistency across the application 4.

The following table provides a summary of the recommended technology stack and its designated roles within the system architecture.

Technology	Primary Role(s)	Justification
Node.js	Core API Server, API Gateway, Notification Handler	Excellent for high-concurrency I/O, real-time communication, and building a scalable backend service 46 75.
Go (Golang)	Scraping Service	Superior performance and concurrency for CPU-intensive, I/O-bound tasks like running headless browsers at scale 74 111.
Python	Analytics Engine, eBay Consumer Service	Rich ecosystem for data analysis (Pandas, NumPy), machine learning, and simplifying API interactions 101.
React	Frontend User Interface	Component-based architecture for building dynamic, real-time dashboards with a rich visual presentation 4 96.
Bash	Infrastructure Scripting, Deployment Automation	Efficient for orchestrating system workflows, managing containers, and automating deployment pipelines.

This polyglot approach leverages the best tool for each job, resulting in a system that is not only powerful and scalable but also easier to develop and maintain in the long run. By carefully assigning responsibilities based on language capabilities, the project can overcome the diverse technical challenges posed by data acquisition, analysis, and real-time user interaction.

Frontend Architecture for Real-Time Market Intelligence Visualization

The frontend of the Pokémon card vendor intelligence system serves as the critical interface through which users interact with the aggregated data and derived insights. Its design must prioritize clarity, interactivity, and, most importantly, real-time responsiveness to meet the user's explicit requirement for a dashboard that displays

information as it happens . To achieve this, the architecture must move beyond traditional page-refresh models and embrace modern web protocols for bidirectional communication. The choice between technologies like Server-Sent Events (SSE) and WebSockets will be pivotal in defining the user experience and the backend implementation complexity.

Server-Sent Events (SSE) emerge as a strong candidate for pushing real-time updates from the server to the client. SSE is a standard that defines how servers can send data to clients automatically. It establishes a one-way communication channel, allowing the server to push updates, such as a newly detected price drop on a watched card or a notification about a relevant marketplace trend, directly to the browser without the client needing to poll for changes [77](#) [78](#) . This protocol is often underrated but powerful for use cases like notifications, newsfeeds, or live commentary [77](#) . From a frontend development perspective, implementing SSE is relatively straightforward. The browser provides a native `EventSource` API, which simplifies the process of establishing and listening to the server connection. When an update is received, the frontend can then intelligently update the user interface, perhaps by animating a new alert in a notification panel or refreshing a specific chart with the latest data point. This approach is less complex to implement than WebSockets and is sufficient for many real-time scenarios where the flow of data is predominantly from the server to the client [79](#) .

While SSE is ideal for server-to-client updates, WebSockets represent the next level of real-time interactivity. WebSockets provide a full-duplex, bidirectional communication channel over a single, long-lived TCP connection [3](#) . This means that both the client and the server can send data at any time. While the primary use case for the dashboard may be receiving updates, there could be scenarios where the client needs to send immediate feedback or commands to the server, such as acknowledging an alert or adjusting filter parameters in real-time. In such cases, WebSockets offer a lower-latency alternative to HTTP requests for bidirectional communication. However, they come with increased complexity in both implementation and server-side management. Given the primary requirement of the user being to *view* real-time data, starting with SSE is a pragmatic and cost-effective approach, with WebSockets as a potential enhancement for future, more interactive features.

The frontend application itself will be built using **React**, leveraging its component-based architecture to create a modular and maintainable codebase [96](#) . The main application shell will consist of a navigation bar, a header for user controls, and a main content area

for the dashboard views. The core of the dashboard will be composed of several key components:

- **Watchlist Dashboard:** A customizable grid or table displaying the user's specified cards. This component will show real-time price data, availability status from various sellers, and highlight significant movements (e.g., green for a price increase, red for a decrease). It will be the primary view for a vendor actively looking for buying opportunities.
- **Trends and Insights Panel:** This component will visualize the output of the Analytics Engine. Using charting libraries compatible with React, it can display line graphs of price history, bar charts showing sale velocity for trending cards, and heatmaps identifying undervalued sets or subsets of cards. This directly addresses the user's desire to see "most trending/uprising/down-trending cards."
- **Real-Time Alerts Feed:** A dedicated section of the UI that acts as a ticker or a modal notification system. Any event that matches a user's criteria—an item listed by a specific seller, a price breach of a watchlist threshold, or a new listing of a trending card—will appear here instantly. This is the direct fulfillment of the real-time alerting requirement.
- **Trading Show Finder:** A map-based or list-based component that pulls data from the Eventbrite API to display upcoming local and national Pokémon trading card events [103](#). This helps vendors plan their physical presence and sourcing strategies.

To manage the state of this complex, real-time application, a state management library like Redux or Context API will be necessary. State management will handle everything from the user's login status and preferences to the currently loaded data for the watchlist and trends panels. The `fetch` API or a library like Axios will be used by React components to communicate with the Node.js backend API server to retrieve initial data on page load and to submit any user-initiated actions (e.g., adding a card to a watchlist). The persistent connection for SSE will be established separately, outside of the standard fetch/polling cycle, to ensure a constant stream of updates. This architecture combines the declarative power of React for UI construction with a robust real-time communication protocol to deliver the dynamic, insightful dashboard that Pokémon card vendors need to stay competitive in a fast-paced market.

Backend Service Design for Automated Data Processing and Alerting

The backend of the Pokémon card vendor intelligence system is the engine room, responsible for orchestrating the entire data lifecycle: from ingestion and processing to analysis and delivery. A well-designed backend, centered around the distributed microservices architecture, ensures that each part of the system performs its specialized function efficiently and reliably. The design must focus on creating autonomous services that communicate asynchronously via a message queue, thereby decoupling data producers from data consumers and preventing systemic bottlenecks.

At the heart of the backend is the **Message Queue**. This acts as the central hub for all data and events flowing through the system. Producers, which are the specialized worker services, will place processed data onto specific queues. For instance, the **Scraping Service** (written in Go) will take raw HTML from Facebook Marketplace, parse the relevant information (card name, price, seller), and then publish a structured JSON object to a "scraped_listings" queue. Simultaneously, the **eBay Consumer Service** (written in Python) will make authenticated calls to the eBay Buy API, receive JSON responses, and publish them to the same queue or a dedicated "ebay_listings" queue. This abstraction allows the downstream consumer services to be completely agnostic to the source of the data; they only care about the format of the message payload.

The primary consumer of these queues is the **Notification Handler Service**, which can be implemented in either Node.js or Python depending on preference. This service subscribes to the data queues and acts as the system's central nervous system. When it receives a new listing event, its first action is to normalize the data, converting it into a consistent internal format regardless of whether it came from a scraper or an official API. Following normalization, it performs a series of checks. It will compare the new listing's price against the watchlists stored in the **Watchlist Management Service**. If a match is found and the price meets the user's specified criteria (e.g., below a certain threshold), the handler will invoke the notification logic. This could involve storing a notification in a database linked to the user's profile and signaling the frontend via the SSE connection. The handler is also responsible for enriching the data; for example, it could call an external service to get the current value of a Pokémon card from a general database or cross-reference a seller's username with their feedback score from the original API response.

Another critical backend service is the **Analytics Engine**, a batch-oriented component, likely written in Python. Unlike the real-time handlers, this service operates on a schedule

(e.g., every night at 2 AM). It connects directly to the **Database Layer**—which holds a historical record of all listings and sales—and performs deep analytical queries. It can calculate moving averages, identify sudden spikes in quantity sold (indicating a rise in popularity), correlate price movements with news events or new game releases, and flag cards that are consistently selling slowly (potential downturns). The results of these analyses—such as a list of the top 10 rising cards of the week or a warning about oversaturation in the Charizard market—would then be stored back in the database, ready to be consumed by the frontend's "Trends and Insights Panel." This service is the technical embodiment of the "identify trending/uprising/down-trending cards" requirement, implemented as a custom algorithm since a Terapeak API is unavailable [51](#).

The **Authentication Service** plays a vital security role. It manages all aspects of user identity, including registration, login, and session management. Critically, it is also the secure repository for API credentials. When a user enters their eBay or TCGplayer API key, this service encrypts and stores it in the database, associating it with the user's account. When the worker services (like the eBay Consumer Service) need to make an API call on behalf of the user, they will request the necessary credentials from the Auth Service, which validates the user's session and returns the token. This prevents credentials from being hardcoded or passed insecurely between services.

Finally, the **API Gateway**, built with Node.js, serves as the public face of the backend. All requests from the React frontend first hit this gateway. It authenticates the user's session, enforces rate limits to protect the backend from abuse, and then routes the request to the appropriate microservice—for example, a request to `/api/watchlists` would be routed to the Watchlist Management Service. This pattern insulates the internal services from direct exposure to the internet, simplifying security and allowing the internal architecture to evolve without impacting the frontend. This comprehensive backend design, with its clear separation of concerns and asynchronous communication, provides the scalable and robust foundation necessary to power a sophisticated, real-time intelligence platform for Pokémon card vendors.

Project Implementation Roadmap and High-Level Structure

The successful realization of the Pokémon card vendor intelligence system requires a structured implementation roadmap that balances ambition with pragmatism. The project should be approached in phases, starting with a Minimum Viable Product (MVP) that

delivers core value and progressively adding more advanced features. This iterative approach allows for early feedback, reduces initial development risk, and ensures that the most critical functionalities are delivered first. A clear, logical project structure is also essential to support this phased development and maintain code quality.

The initial phase should focus on building the core data ingestion and real-time alerting capabilities, centered around the most accessible and reliable data source: eBay. The first step is to conduct thorough research into the eBay Buy API, Inventory API, and especially the Notification API [15](#) [41](#) [92](#). A proof-of-concept (PoC) should be developed to validate the end-to-end flow: authenticating a user via OAuth2, having the user provide their API keys, and successfully subscribing to eBay notifications for a test listing [9](#) [40](#). This PoC will serve as a blueprint for the full **Notification Handler Service**. During this phase, the basic frontend dashboard powered by React should be built, featuring a watchlist component and a simple alerts feed that can receive messages pushed via SSE from the backend.

Once the eBay integration is stable, the second phase should tackle the next most important data source: TCGplayer. Similar to the eBay PoC, a new service (**TCGplayer Consumer Service**) must be developed to interact with its API. The frontend will be extended to display listings from TCGplayer alongside those from eBay, and the alerting logic will be updated to monitor for price changes and new listings across both platforms. This phase directly expands the system's market coverage and provides greater value to vendors who trade on multiple platforms.

With a solid foundation for API-based data ingestion in place, the third phase can address the more challenging problem of scraping. A PoC using Playwright should be created to reliably extract data from a sample Facebook Marketplace page [93](#). This will help quantify the technical challenges and resource requirements of browser automation. Once the scraping methodology is proven, the **Scraping Service** (in Go) and its corresponding frontend components can be developed. This phase completes the system's ability to monitor the three primary marketplaces specified by the user.

The final phase of the MVP involves developing the analytical and discovery features. The **Analytics Engine** (in Python) can be built to perform batch analysis on the historical data now being collected. Initial versions might focus on simple metrics like average sale price over time, with more sophisticated trend detection algorithms added later. Concurrently, an integration with the **Eventbrite API** should be implemented to populate the "upcoming trading shows" feature, providing vendors with valuable logistical planning information [103](#).

This structure provides a clean and scalable foundation. The `client` directory contains the React application. The `server` directory hosts the core Node.js application. The `services` directory is the heart of the distributed system, with subdirectories for each specialized worker, reflecting the chosen technology stack. Finally, the `database` directory holds scripts for managing the database schema and initial data. By following this phased roadmap and adhering to this structural organization, the development team can systematically build a powerful, comprehensive, and highly valuable tool for the competitive world of Pokémon card trading.

Reference

1. The 3 Biggest Trends in Backend Development (2024-2026) https://dev.to/yodev_grego/the-3-biggest-trends-in-backend-development-2024-2026-24co
2. [PDF] AMENDMENT N° 1 WORK PROGRAMME 2025-2026 | Europe's Rail https://rail-research.europa.eu/wp-content/uploads/2025/06/GB-Decision_02-25_WP2025-2026_amendment1_Clean-1.pdf
3. SSE, WebSockets, or Polling? Build a Real-Time Stock App ... - Dev.to <https://dev.to/itaybenami/sse-websockets-or-polling-build-a-real-time-stock-app-with-react-and-hono-1h1g>
4. Build Real-Time Stock Price Dashboard in ReactJS (Code + GitHub) <https://www.sevensquaretech.com/reactjs-live-stock-price-dashboard-websocket-github-code/>
5. Celebrating Music with Tanya McCole on Highland Radio - TikTok <https://www.tiktok.com/@highlandradio/video/7298771077096918305>
6. i had issue regarding access token apis - The eBay Community <https://community.ebay.com/t5/Token-Messaging-Sandbox-related/i-had-issue-regarding-access-token-apis/td-p/34161817>
7. Say Good bye to File Exchange - Find of I guess..... <https://community.ebay.com/t5/Seller-Tools/Say-Good-bye-to-File-Exchange-Find-of-I-guess/td-p/31786091>
8. Sample PHP for generating and renewing a Shopping token <https://community.ebay.com/t5/Token-Messaging-Sandbox-related/Sample-PHP-for-generating-and-renewing-a-Shopping-token/td-p/33540297>
9. Ebay Oauth2 Dev account-Seller Account link - The eBay Community <https://community.ebay.com/t5/Seller-Tools/Ebay-Oauth2-Dev-account-Seller-Account-link/td-p/31928600>

10. The legacy JS API is deprecated and will be removed in Dart Sass ... <https://stackoverflow.com/questions/78997907/the-legacy-js-api-is-deprecated-and-will-be-removed-in-dart-sass-2-0-0>
11. Playwright vs Selenium : What to Choose in 2026 - HeadSpin <https://www.headspin.io/blog/selenium-vs-playwright-comparison>
12. Notification API Overview | eBay Developers Program <https://developer.ebay.com/api-docs/buy/notification/overview.html>
13. FixedPriceTransaction - eBay Developers Program https://developer.ebay.com/api-docs/static/pn_fixed-price-transaction.html
14. eBay Developers Program Q3 2025 Newsletter https://developer.ebay.com/updates/newsletter/q3_2025
15. Inventory API Overview | eBay Developers Program <https://developer.ebay.com/api-docs/sell/inventory/overview.html>
16. Platform Notifications | eBay Developers Program <https://developer.ebay.com/api-docs/static/platform-notifications-landing.html>
17. eBay Developers Program Q4 2025 Newsletter https://developer.ebay.com/updates/newsletter/q4_2025
18. ItemListed - eBay Developers Program https://developer.ebay.com/api-docs/static/pn_item-listed.html
19. eBay Developers Program Q2 2025 Newsletter https://developer.ebay.com/updates/newsletter/q2_2025
20. Browse API resources - eBay Developers Program <https://developer.ebay.com/api-docs/buy/browse/resources/methods>
21. search: eBay Browse API - eBay Developers Program https://developer.ebay.com/api-docs/buy/browse/resources/item_summary/methods/search
22. Sitemap - Red Hat Ecosystem Catalog <https://catalog.redhat.com/en/sitemap/listings>
23. 116186-WP-AlternativeFinanceReportlowres-PUBLIC.txt <https://documents1.worldbank.org/curated/en/701331497329509915/txt/116186-WP-AlternativeFinanceReportlowres-PUBLIC.txt>
24. User Guide for AsyncOS 15.2 for Cisco Secure Web Appliance <https://www.cisco.com/c/en/us/td/docs/security/wsa/wsa-15-2/user-guide/swa-userguide-15-2/m-access-control.html>
25. Impact Assessment - European Union <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:52018SC0138>
26. Why Do Pokémon Cards Have Investment Value After a ... - 富途资讯 <https://news.futunn.com/en/post/62286159/why-do-pok-mon-cards-have-investment-value-after-a>

27. Gen Z, Pokémon Cards & Boy Math: Can They Beat the S&P 500? <https://www.archivemarketresearch.com/news/article/gen-z-pokmon-cards-boy-math-can-they-beat-the-sp-500-75242>
28. Finding API - The eBay Community <https://community.ebay.com/t5/Traditional-APIs-Selling/Finding-API/td-p/34515870>
29. Issue with the FindingService API when using findI... <https://community.ebay.com/t5/Traditional-APIs-Search/Issue-with-the-FindingService-API-when-using-findItemsAdvanced/td-p/34970218>
30. Market Insights API Application - The eBay Community <https://community.ebay.com/t5/eBay-APIs-Talk-to-your-fellow/Market-Insights-API-Application/td-p/34065439>
31. Real-Time MLOps Pipeline for Market Monitoring with FastAPI and ... https://www.linkedin.com/posts/abhay-kumar-a1b098242_mlops-dataengineering-fastapi-activity-7429513225609605120-fWkX
32. Designing a Distributed E-commerce Marketplace with Microservices https://www.linkedin.com/posts/mohan-nayak-49954392_%F0%9D%97%9B%F0%9D%97%BC%F0%9D%98%84-%F0%9D%98%81%F0%9D%97%BC-%F0%9D%97%97%F0%9D%97%B2%F0%9D%98%80%F0%9D%97%B6%F0%9D%97%B4%F0%9D%97%BB-%F0%9D%97%AE-%F0%9D%97%97%F0%9D%97%B6%F0%9D%98%80%F0%9D%98%81%F0%9D%97%BF%F0%9D%97%B6%F0%9D%97%AF%F0%9D%98%82%F0%9D%98%81%F0%9D%97%B2%F0%9D%97%B1-activity-7415804718779715584-bQEo
33. Meta Policy and Ads Update 2026 | PDF | Artificial Intelligence - Scribd <https://www.scribd.com/document/1002367609/Meta-Policy-and-Ads-Update-2026>
34. Community Calendar - City of Auburn https://www.auburnwa.gov/city_hall/communitycalendar
35. Get all active listings using eBay API <https://community.ebay.com/t5/Traditional-APIs-Orders/Get-all-active-listings-using-eBay-API/td-p/34030058>
36. getItem: eBay Browse API - eBay Developers Program <https://developer.ebay.com/api-docs/buy/browse/resources/item/methods/getItem>
37. GetOrders - API Reference - Trading API - eBay Developers Program <https://developer.ebay.com/devzone/xml/docs/reference/ebay/getorders.html>
38. Meet the buy apis let your visitors shop purchase and track orders ... <https://partnernetwork.ebay.com/solutions/meet-the-buy-apis-let-your-visitors-shop-purchase-and-track-orders-without-ever-leaving-your-site>
39. where I can find eBay API for listing item? <https://community.ebay.com/t5/Ask-a-Mentor/where-I-can-find-eBay-API-for-listing-item/td-p/33966782>

40. Get started with eBay APIs - eBay Developers Program https://developer.ebay.com/api-docs/static/gs_ebay-rest-getting-started-landing.html
41. Alert! - Finding API and Shopping API to be decommissioned in 2025 <https://community.ebay.com/t5/Traditional-APIs-Search/Alert-Finding-API-and-Shopping-API-to-be-decommissioned-in-2025/td-p/34222062>
42. eBay Developers Program Q4 2023 Newsletter https://developer.ebay.com/updates/newsletter/q4_2023
43. which API to get all inventory active listing item... <https://community.ebay.com/t5/RESTful-Sell-APIs-Account/which-API-to-get-all-inventory-active-listing-items/td-p/34748650>
44. Please help me, I'm hitting the rate limit without... <https://community.ebay.com/t5/Traditional-APIs-Search/Please-help-me-I-m-hitting-the-rate-limit-without-even-one-call/td-p/35025967>
45. eBay APIs: Talk to your fellow developers <https://community.ebay.com/t5/eBay-APIs-Talk-to-your-fellow/bd-p/eBayAPIs-Talk-to-your-fellow-developer>
46. Rust vs Node.js vs Go: Performance Comparison for Backend ... <https://dev.to/hamzakhan/rust-vs-nodejs-vs-go-performance-comparison-for-backend-development-2g69>
47. Performance Comparison of GoLang, Node.js, and PHP - LinkedIn <https://www.linkedin.com/pulse/technology-business-use-case-performance-comparison-golang-rxjlf>
48. Rust vs Go vs Bun vs Node.js: The Ultimate 2024 Performance ... <https://dev.to/hamzakhan/rust-vs-go-vs-bun-vs-nodejs-the-ultimate-2024-performance-showdown-2jml>
49. Building A Pokémon Card Scanning | PDF - Scribd <https://www.scribd.com/document/890806272/Building-a-Poke-mon-Card-Scanning>
50. Best Way To Sell Magic: The Gathering Cards: 2026 Data Strategies <https://www.alibaba.com/product-insights/best-way-to-sell-magic-the-gathering-cards-2026-data-strategies.html>
51. Terapeak API Documentation - The eBay Community <https://community.ebay.com/t5/Seller-Tools/Terapeak-API-Documentation/td-p/32138281>
52. Accessing Terapeak Product Research Information <https://community.ebay.com/t5/Traditional-APIs-Search/Accessing-Terapeak-Product-Research-Information/td-p/33827092>
53. Get data of terapeak research using API - The eBay Community <https://community.ebay.com/t5/Traditional-APIs-Search/Get-data-of-terapeak-research-using-API/td-p/33565469>

54. How to use APIs to get Terapeak information in Python? <https://community.ebay.com/t5/eBay-APIs-Talk-to-your-fellow/How-to-use-APIs-to-get-Terapeak-information-in-Python/td-p/34502508>
55. Leveraging SNS Data for E-Sports Recommendation: Analyzing ... <https://www.mdpi.com/2079-9292/14/1/94>
56. (PDF) Finally! My first shiny!": Assessing Text Mining Tools and ... <https://www.academia.edu/92799240/>
Finally_My_first_shiny!":_Assessing_Text_Mining_Tools_and_Methodologies_for_the_Study_of_the_Pok%C3%A9mon_Sword_and_Shield_Twitter_Online_Community
57. Updates - eBay Developers Program <https://developer.ebay.com/updates>
58. eBay Developers Program <https://developer.ebay.com/develop>
59. eBay Developers Program Q1 2025 Newsletter https://developer.ebay.com/updates/newsletter/q1_2025
60. API Status - eBay Developers Program <https://developer.ebay.com/support/api-status>
61. publishOffer: eBay Inventory API - eBay Developers Program <https://developer.ebay.com/api-docs/sell/inventory/resources/offer/methods/publishOffer>
62. Can the Transparency and Consent Framework Ever Authorise Real ... https://www.researchgate.net/publication/358550380_Impossible_Askes_Can_the_Transparency_and_Consent_Framework_Ever_Authorise_Real-Time_Bidding_After_the_Belgian_DPA_Decision
63. Course 1 - Big Data Ecosystem | PDF | Apache Hadoop - Scribd <https://www.scribd.com/document/514822243/Course-1-Big-Data-Ecosystem>
64. GitHub - Toddmotto:Public-Apis: A Collective List of Public JSON ... <https://www.scribd.com/document/385605188/GitHub-Toddmotto-Public-Apis-A-Collective-List-of-Public-JSON-APIs-for-Use-in-Web-Development>
65. Pokémon GO Forensics: An Android Application Analysis - MDPI <https://www.mdpi.com/2078-2489/8/3/71>
66. Xin-She Yang Simon Sherratt Nilanjan Dey Amit Joshi Editors <https://link.springer.com/content/pdf/10.1007/978-981-96-6432-0.pdf>
67. Anael BEUNAICHE - Consulting Engineer at Aubay France | LinkedIn <https://fr.linkedin.com/in/anael-beunaiche-205b4012a/en>
68. [PDF] HCI in Games - ResearchGate https://www.researchgate.net/profile/Olivier-Gavarry/publication/372229826_Escapad_Dance_Serious_Game_Designing_a_Therapeutic_Tool_for_Elderly_with_Mild_Cognitive_Impairment/links/64ec9fb340289f7a0fbffb0a/Escapad-Dance-Serious-Game-Designing-a-Therapeutic-Tool-for-Elderly-with-Mild-Cognitive-Impairment.pdf

69. TowardsDataScience-博客中文翻译-2019-七十一 - 绝不原创的飞龙 <https://www.cnblogs.com/apachecn/p/18462404>
70. OneNote Basics: Efficiently Set Up Your Notebook | TikTok <https://www.tiktok.com/@imperfectplannermom/video/7477737700515138821>
71. [PDF] FALL 2025 / WINTER 2026 - Kirkland, WA <https://www.kirklandwa.gov/files/sharedassets/public/v/4/parks-amp-comm-services/recreation/rec-guide/25-fall-winter-26-activity-guide-final-10-13-2025-optimized.pdf>
72. [PDF] Parent Handbook 2026 - Amazon AWS https://bzmt-cdn.s3.amazonaws.com/BZMT_Parent_Handbook.pdf
73. Rust vs Go vs Node: Performance Showdown | Rajat Gupta posted ... https://www.linkedin.com/posts/kkanhiya_go-vs-rust-vs-node-1-million-requests-later-activity-7420881380558991360-HJmd
74. Go vs Node.js: Which Backend Powerhouse Will Dominate in 2024? <https://dev.to/hamzakhan/go-vs-nodejs-which-backend-powerhouse-will-dominate-in-2024-oe2>
75. Backend 2025: Node.js vs Python vs Go vs Java - Talent500 <https://talent500.com/blog/backend-2025-nodejs-python-go-java-comparison/>
76. Server-Sent Events vs WebSockets – How to Choose a Real-Time ... <https://www.freecodecamp.org/news/server-sent-events-vs-websockets/>
77. Real-Time Updates in Web Apps: Why I Chose SSE Over WebSockets <https://dev.to/okrahul/real-time-updates-in-web-apps-why-i-chose-sse-over-websockets-k8k>
78. Server-Sent Events vs WebSockets: Real-Time Tech Explained https://www.linkedin.com/posts/huseyinbabal_server-sent-events-sse-build-a-real-time-activity-7432150602471723008-yk9f
79. WebSockets Vs Server-Sent Events (SSE) - Ably <https://ably.com/blog/websockets-vs-sse>
80. simplest way to create listings on sandbox - The eBay Community <https://community.ebay.com/t5/RESTful-Sell-APIs-Account/simplest-way-to-create-listings-on-sandbox/td-p/33532334>
81. What is Computer Programming | PDF - Scribd <https://www.scribd.com/document/999480581/What-is-Computer-Programming>
82. tm2113258-19_424b4 - none - 31.4845669s - SEC.gov https://www.sec.gov/Archives/edgar/data/1454938/000110465921095449/tm2113258-19_424b4.htm
83. [PDF] Minutes of Council Meeting held on Tuesday 26 August 2025, 5.30PM <https://www.mandurah.wa.gov.au/-/media/files/com/city-and-council/council/council-and-committee-meetings/agendas-and-minutes/2025/council/council-meeting-minutes-26-aug-2025.pdf>
84. True lies in geospatial big data: detecting location spoofing in social ... <https://www.tandfonline.com/doi/full/10.1080/19475683.2017.1280536>

85. An analysis of augmented reality features in location-based games <https://journals.sagepub.com/doi/10.1177/13548565231156495>
86. [PDF] csi-logistics-trend-radar-6-dhl.pdf <https://www.dhl.com/content/dam/dhl/global/csi/documents/pdf/csi-logistics-trend-radar-6-dhl.pdf>
87. Our first week was too much fun! Pumped for next week ... - TikTok <https://www.tiktok.com/@merchtcg/video/6988256937477426437>
88. Notification API Release Notes - eBay Developers Program <https://developer.ebay.com/api-docs/sell/notification/release-notes.html>
89. NotificationEventTypeCodeType - API Reference - Trading API <https://developer.ebay.com/devzone/xml/docs/reference/ebay/types/NotificationEventTypeCodeType.html>
90. getItem - browse API - eBay Developers Program <https://developer.ebay.com/api-docs/buy/browse/resources/item/methods/getItem>
91. Availability: eBay Inventory API <https://developer.ebay.com/api-docs/sell/inventory/types/slr:Availability>
92. Notification API resources - eBay Developers Program <https://developer.ebay.com/api-docs/commerce/notification/resources/methods>
93. 打造绕过全维度前端反爬的无头浏览器爬虫（2026实战版） https://blog.csdn.net/shanwei_spider/article/details/157611866
94. What's new in Red Hat OpenShift <https://www.redhat.com/zh/whats-new-red-hat-openshift>
95. [PDF] Metadata API Developer Guide - Salesforce https://resources.docs.salesforce.com/latest/latest/en-us/sfdc/pdf/api_meta.pdf
96. The Complete Full-Stack Developer Roadmap for 2026 <https://dev.to/thebitforge/the-complete-full-stack-developer-roadmap-for-2026-2i0j>
97. Staff Software Engineer, Vehicle Configurator, Digital Experience <https://www.tesla.com/careers/search/job/staff-software-engineer-vehicle-configurator-digital-experience-263297>
98. [XML] <https://stackoverflow.com/sitemap-questions-140.xml> <https://stackoverflow.com/sitemap-questions-140.xml>
99. PC Pro - Issue 367 April 2025 | PDF | Personal Computers - Scribd <https://www.scribd.com/document/911801063/PC-Pro-Issue-367-April-2025>
100. eBay APIs: SDKs <https://community.ebay.com/t5/eBay-APIs-SDKs/bd-p/APIs-SDKs>
101. Christian Kästner :: Publications - CMU School of Computer Science <https://www.cs.cmu.edu/~ckaestne/publications.html>
102. rddt-20241231 - SEC.gov <https://www.sec.gov/Archives/edgar/data/1713445/000171344525000018/rddt-20241231.htm>

- [illegible]